Contents

1. Introduction

The integration of artificial intelligence into live surveillance systems has transformed passive monitoring into active, data-driven analytics. However, applying complex deep learning models directly to high-resolution video streams in real-time presents significant computational challenges.

1.1 Problem Statement

Applying cascading neural networks (detection, facial extraction, and classification) on a single thread causes severe latency, making real-time monitoring impossible. A synchronous approach forces the video stream to wait for the heaviest model to finish predicting before moving to the next frame. There is a need for a decoupled, edge-optimized architecture where lightweight tracking and heavyweight classification can occur at different frequencies without interrupting the visual feed.

1.2 System Objectives

- **Continuous Tracking:** Implement YOLOv8 with ByteTrack to maintain consistent bounding boxes and unique IDs for individuals across frames.
- **Asynchronous Classification:** Utilize a Thread and Queue-based architecture to offload gender classification to a background worker.
- **Dynamic Visual Overlay:** Display live confidence scores, bounding boxes, system health (AI errors), and inference times directly on the RTSP video feed.

1.3 Scope of the System

The system is scoped to process an RTSP (Real-Time Streaming Protocol) video stream. It identifies "person" classes, isolates their upper-body/facial region, and classifies them into a binary demographic (man/woman). The system is designed for edge-computing environments and prioritizes stream continuity over instant classification.

---

2. System Requirements Specification

2.1 Functional Requirements

- **Video Ingestion:** The system must read frames continuously from a secured RTSP IP camera stream.
- **Subject Isolation:** The system must crop the detected person's bounding box and attempt to extract facial features using DeepFace. If a face is unreadable, it must intelligently fallback to analyzing the upper 40% of the bounding box.
- **State Management:** The system must maintain a global_registry to remember the classification results of previously analyzed track IDs so that the classification model only runs once per person.

2.2 Non-Functional Requirements

- **Performance:** The primary YOLOv8 tracking loop must execute within **40-60ms**. The background classification task must complete within **300-600ms** per subject.
- **Reliability:** The background thread must gracefully catch execution errors (e.g., failed model inference) and assign an "AI Error" tag rather than crashing the system.
- **Resource Management:** The analysis queue must be capped (maxsize=20) to prevent memory overflow during high-traffic events.

2.3 Hardware & Software Constraints

- **Programming Language:** Python 3.x
- **Deep Learning Frameworks:** TensorFlow/Keras (Custom gender model), PyTorch (YOLOv8 framework).
- **Computer Vision Libraries:** OpenCV (cv2), DeepFace, Ultralytics.
- **Concurrency:** Python native threading and queue libraries.

---

3. System Architecture & High-Level Design

3.1 Concurrent Processing Architecture

The system utilizes a modified Producer-Consumer architecture to achieve Multi-Access Edge Computing (MEC) optimization.

1. **RTSP Stream Thread:** Dedicated exclusively to pulling the latest frame into a buffer, ensuring the application always reads the most current physical moment, ignoring network lag.
2. **Main Application Thread (Producer):** Runs the YOLOv8 model. It identifies bounding boxes, draws UI elements, and pushes cropped images of *newly detected* individuals into the Analysis Queue.
3. **Classification Worker Thread (Consumer):** Continuously monitors the queue. It pops a cropped image, runs RetinaFace extraction, normalizes the tensor, executes the Keras gender classifier, and updates the shared Global Registry.

3.2 Component Workflow

- **Detection:** Frame -> YOLOv8 -> ByteTrack IDs.
- **Routing:** * *If ID is known:* Fetch label from Global Registry -> Draw Green/Red Box.
    - *If ID is new:* Draw Yellow Box ("Analyzing...") -> Send to Queue.
- **Classification:** Queue -> DeepFace Extract -> Resize (96x96 RGB) -> Keras Model -> Update Registry.

---

4. Detailed Implementation & Design

4.1 RTSP Ingestion & Tracking

The RTSPStream class initializes a cv2.VideoCapture object with a restricted buffer size (CAP_PROP_BUFFERSIZE = 1). A background daemon thread continually flushes the buffer, ensuring the main thread always accesses the absolute latest frame. Tracking is handled by YOLO (yolov8n.pt) configured with the bytetrack.yaml tracker, enforcing a minimum confidence threshold of 0.4.

4.2 Asynchronous Classification Pipeline

When a valid bounding box (width > 20px, height > 50px) is identified as a new ID, it is added to the analysis_queue. The classification_worker thread processes these crops by:

1. **Facial Extraction:** Attempting to isolate the face using retinaface.
2. **Fallback Heuristic:** If a face is not detected (e.g., subject is turned away), the system falls back to taking the top 40% of the bounding box crop to evaluate contextual features (hair, clothing).
3. **Tensor Normalization:** The crop is resized to 96x96, forced into a 3-channel RGB format, and normalized (divided by 255.0).
4. **Prediction:** The tensor is passed to efficient_gender_model.keras, returning a confidence array mapped to the ['man', 'woman'] classes.

5. Results & Performance Evaluation

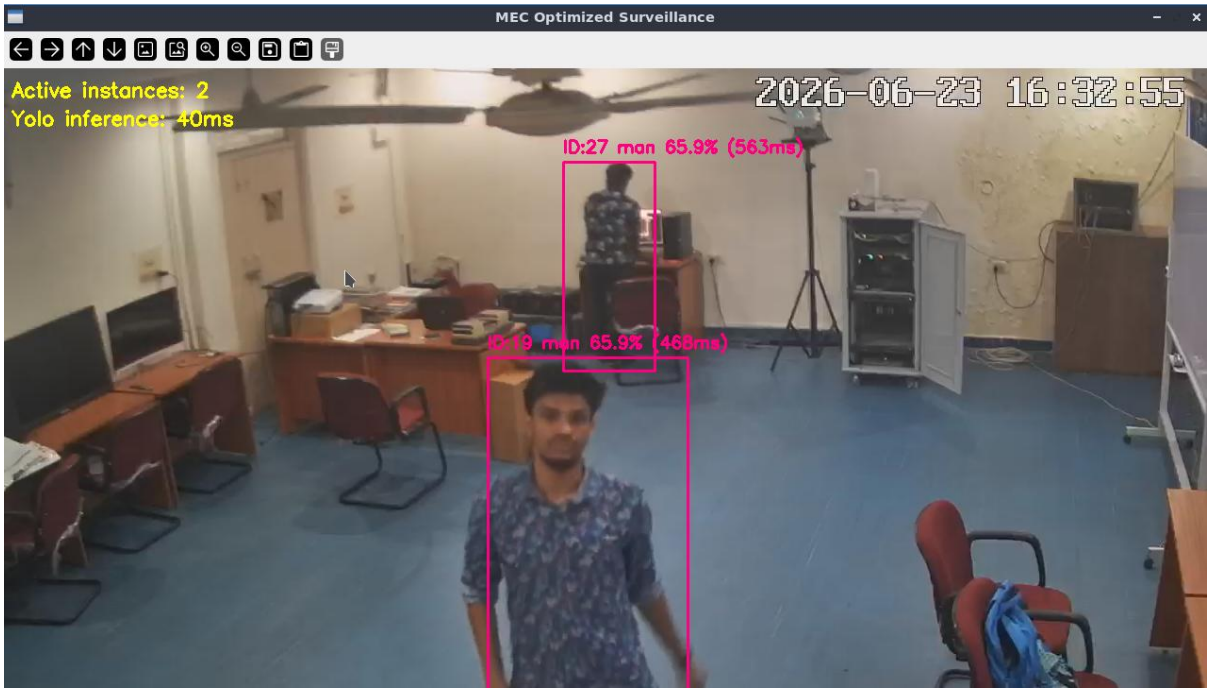
5.1 Real-Time Metrics

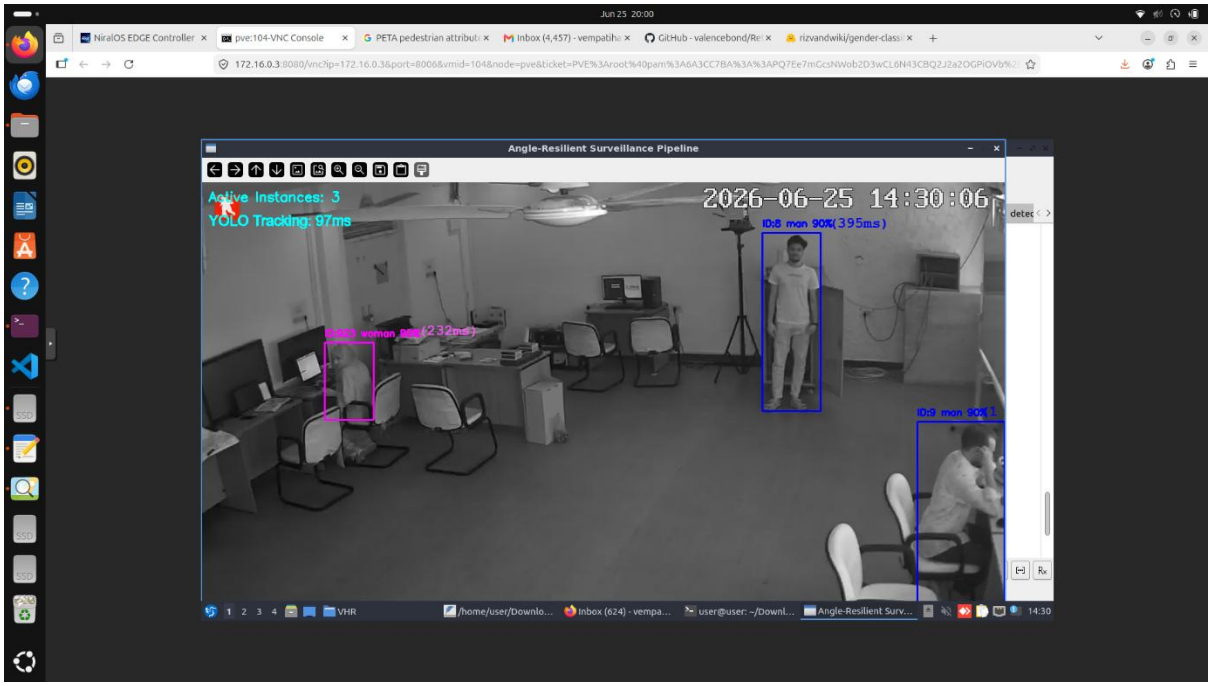
The multi-threaded architecture successfully prevents the heavy Keras model from bottlenecking the video feed. During execution, the system maintains high tracking fluidity while classifications populate asynchronously.

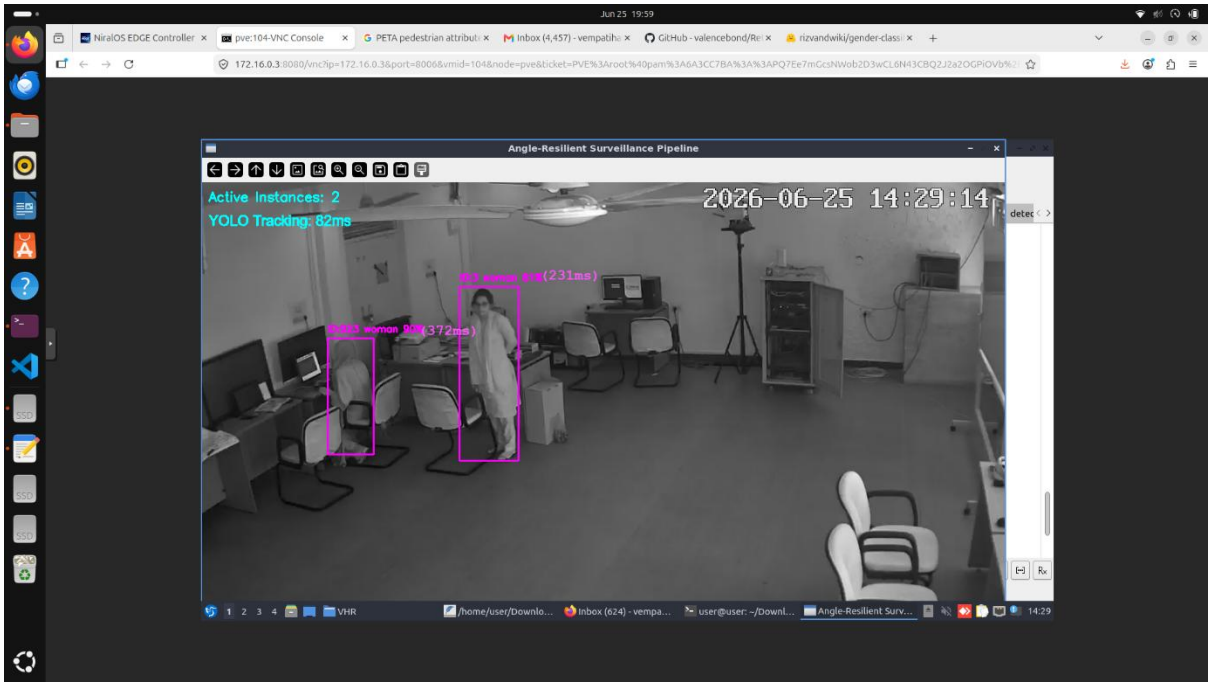
- **Detection & Tracking Latency (YOLOv8 + ByteTrack):** Maintained a consistent inference time of **40ms to 60ms** per frame.
- **Classification Latency (DeepFace + Keras):** The asynchronous worker thread processed individual demographic classifications within a timeframe of **300ms to 600ms** per subject.
- **UI Updates:** Upon successful classification, the tracking box immediately dynamically updates from Yellow ("Analyzing...") to Green/Red with the corresponding confidence percentage.

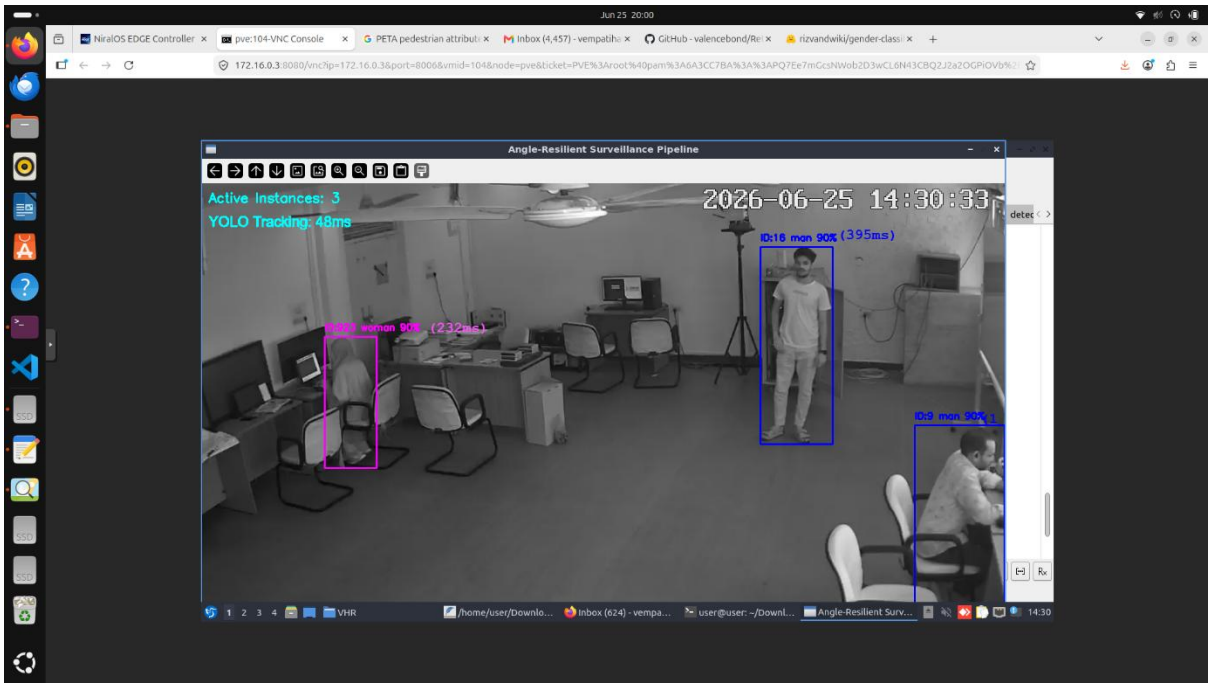
5.2 Visual Output

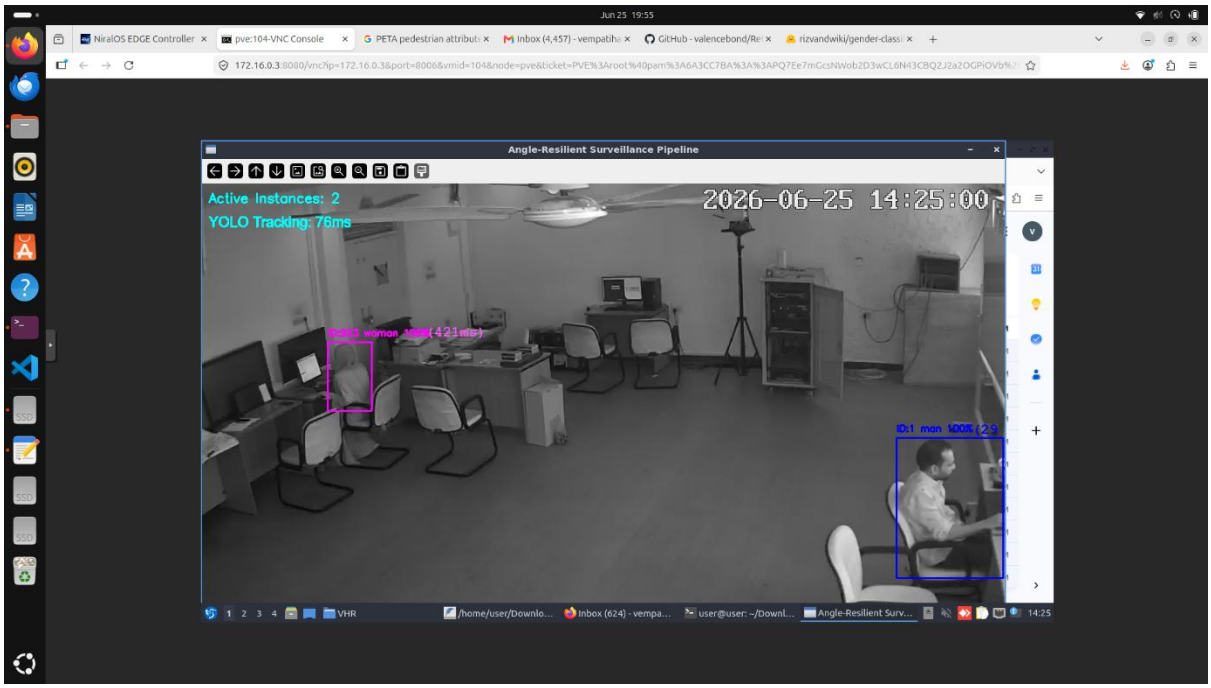
Including Inference Time:

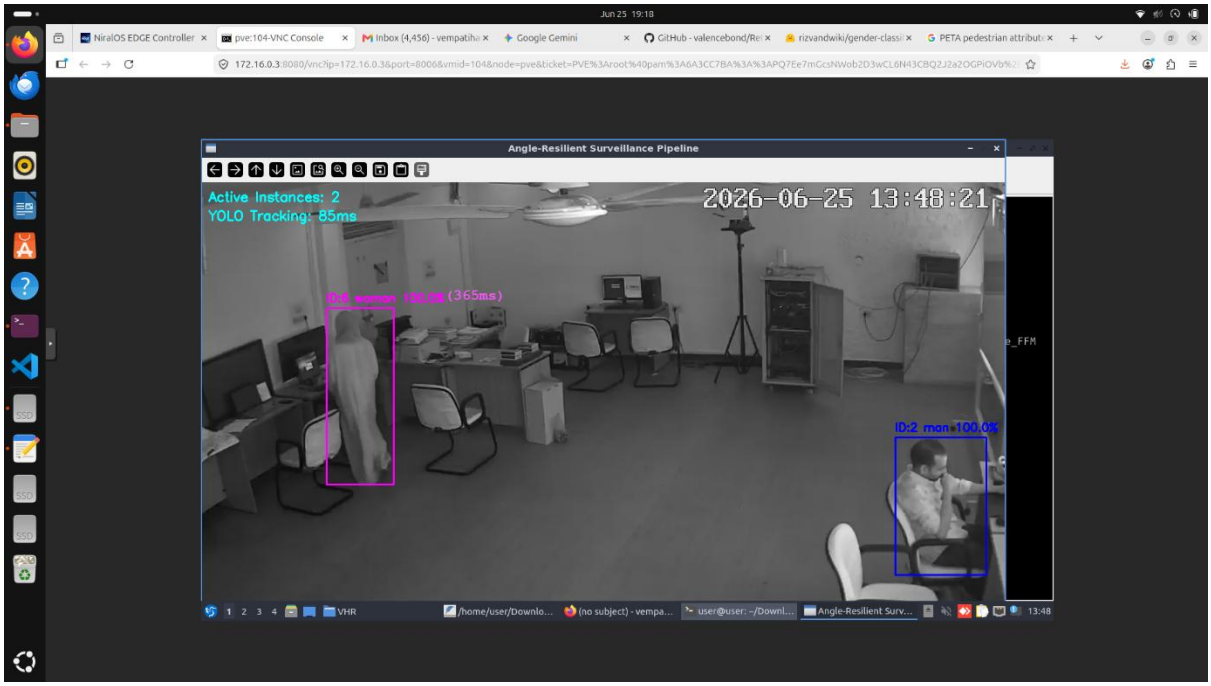

MEC Optimized Surveillance
Active instances: 2
Yolo inference: 40ms
2026-06-23 16:32:55
ID:27 man 65.9% (563ms)
ID:19 man 65.9% (468ms)

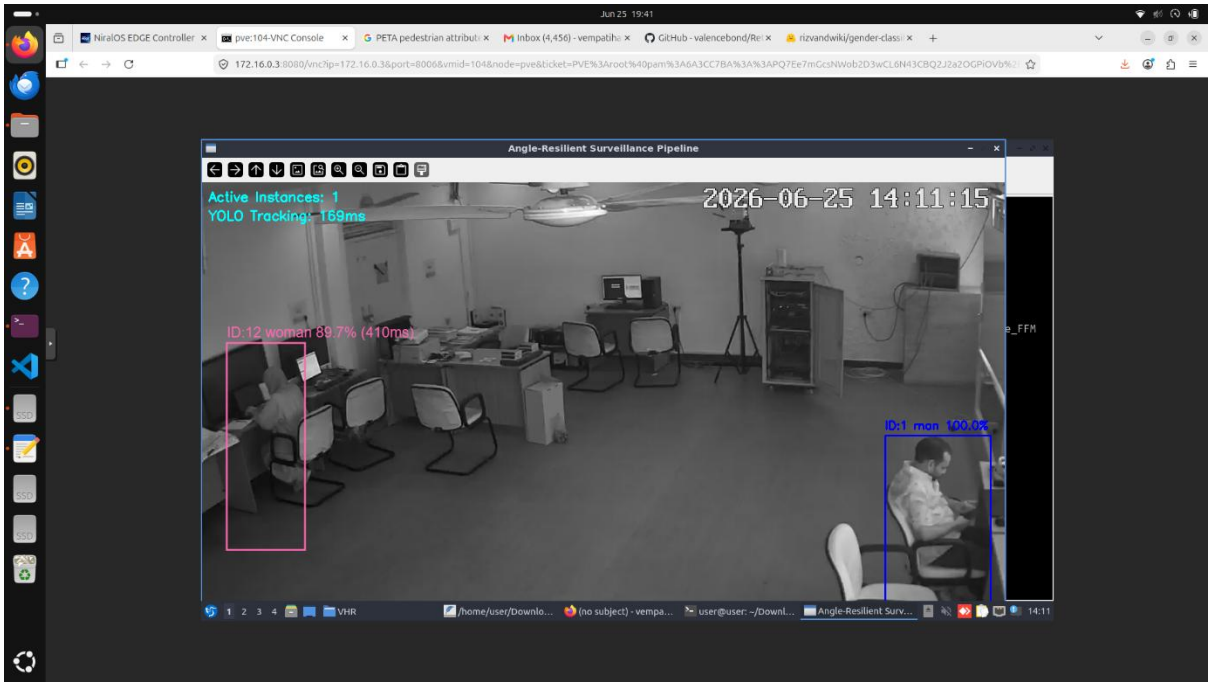

Angle-Resilient Surveillance Pipeline
Active Instances: 3
YOLO Tracking: 97ms
2026-06-25 14:30:06
ID:8 man 90%(395ms)


Jun 25 19:59
Angle-Resilient Surveillance Pipeline
Active Instances: 2
YOLO Tracking: 82ms
2026-06-25 14:29:14
ID:3 woman 89%(231ms)
ID:23 woman 90%(372ms)
14:29

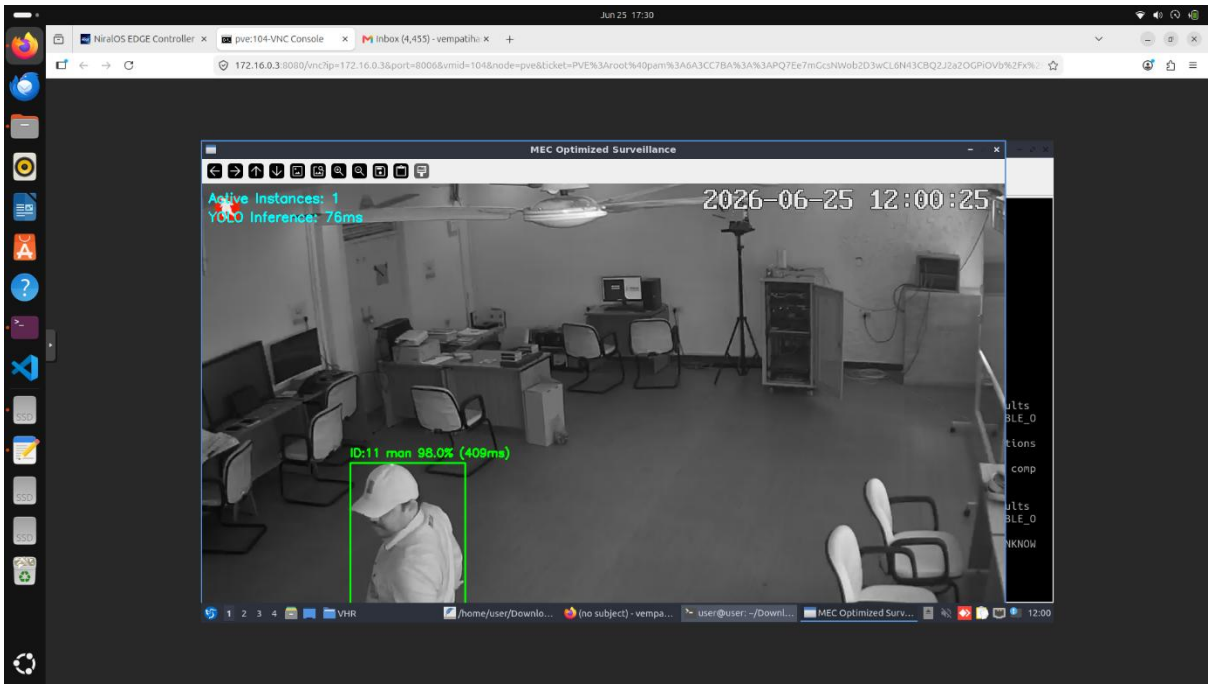

Jun 25 20:00
Angle-Resilient Surveillance Pipeline
Active Instances: 3
YOLO Tracking: 48ms
2026-06-25 14:30:33
ID:18 man 90% (395ms)
ID:23 woman 90% (232ms)
ID:9 man 90%
14:30

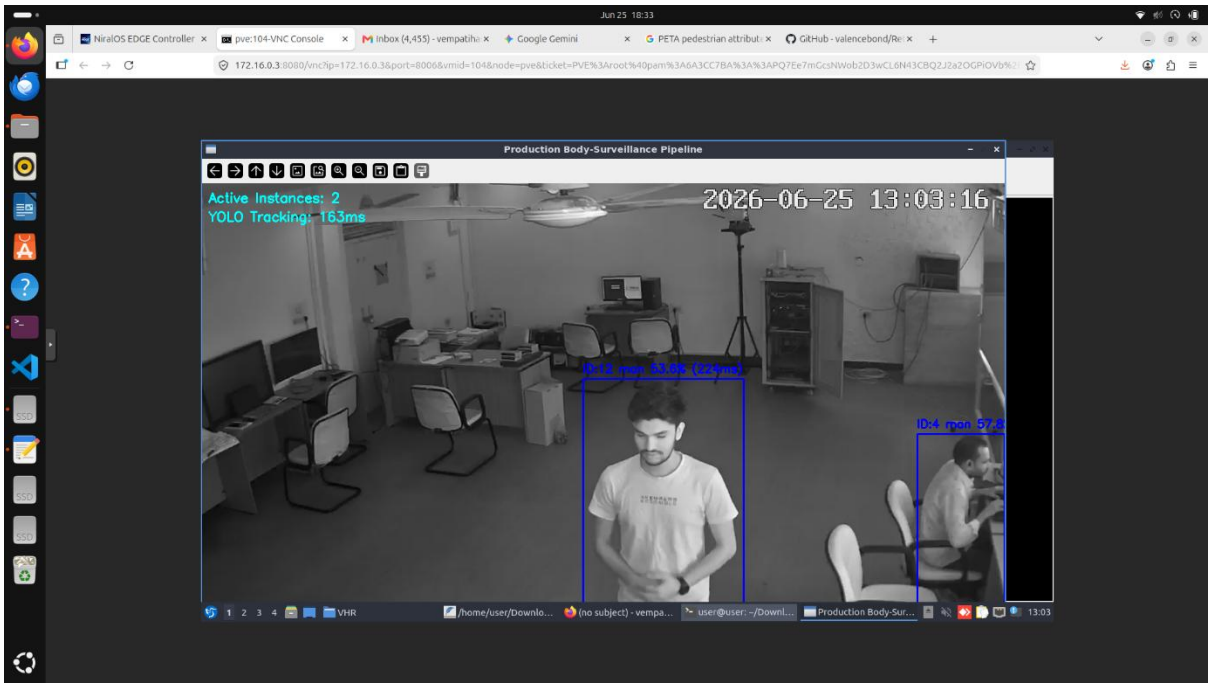

Angle-Resilient Surveillance Pipeline
Active Instances: 2
YOLO Tracking: 76ms
2026-06-25 14:25:00
ID:1 man 100% (421ms)
ID:2 man 100%

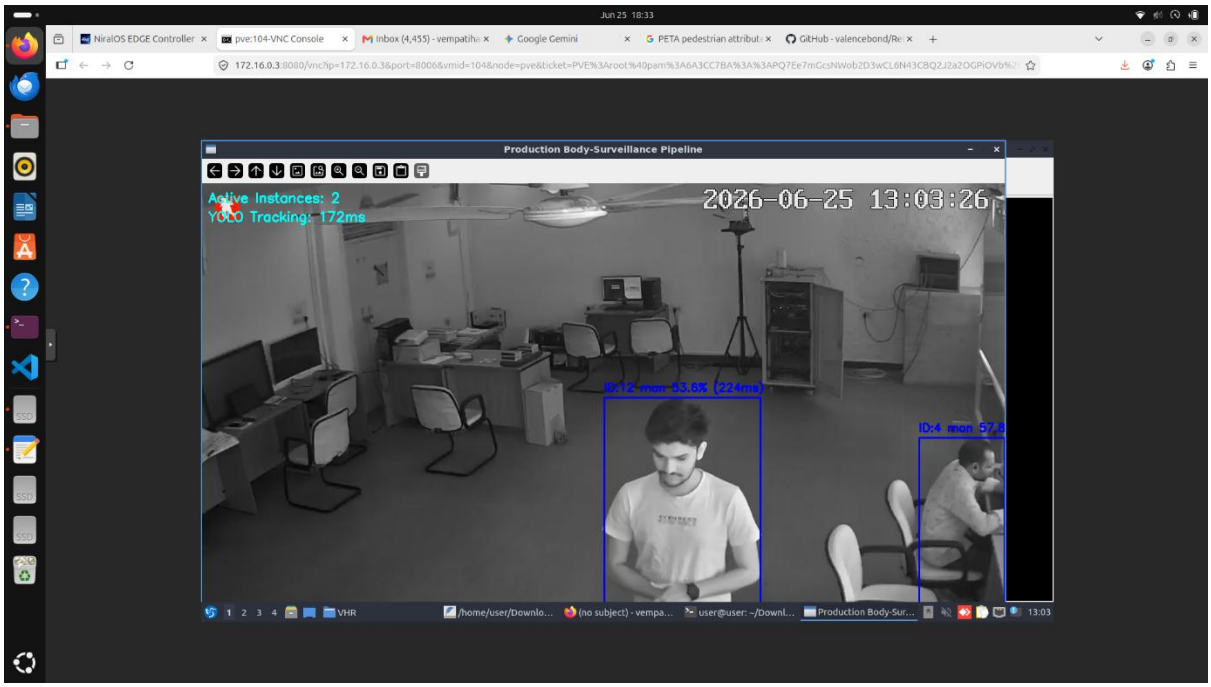

Angle-Resilient Surveillance Pipeline
Active Instances: 2
YOLO Tracking: 85ms
2026-06-25 13:48:21
ID:1 woman 100.0% (365ms)
ID:2 man 100.0%


Angle-Resilient Surveillance Pipeline
Active Instances: 1
YOLO Tracking: 169ms
2026-06-25 14:11:15
ID:12 woman 89.7% (410ms)
ID:1 man 100.0%


MEC Optimized Surveillance
Active Instances: 1
YOLO Inference: 59ms
2026-06-25 12:00:58
ID:11 man 98.0% (409ms)
ID:12 woman 89.7% (410ms)


Jun 25 17:30
NiralOS EDGE Controller
pve:104-VNC Console
Inbox (4,455) - vempatiha
MEC Optimized Surveillance
Active Instances: 1
YOLO Inference: 76ms
2026-06-25 12:00:25
ID:11 man 98.0% (409ms)
VHR
/home/user/Downlo...
(no subject) - vempa...
user@user: ~/Downl...
MEC Optimized Surv...
12:00


Jun 25 18:33
NiralOS EDGE Controller
pve:104-VNC Console
Inbox (4,455) - vempatiha
Google Gemini
PETA pedestrian attribut
GitHub - valencebond/Re
Production Body-Surveillance Pipeline
Active Instances: 2
YOLO Tracking: 163ms
2026-06-25 13:03:16
ID:4 man 57.8
VHR
/home/user/Downlo...
(no subject) - vempa...
user@user: ~/Downl...
Production Body-Sur...
13:03


Jun 25 18:33
NiralOS EDGE Controller
pve:104-VNC Console
Google Gemini
GitHub - valencebond/Re
Production Body-Surveillance Pipeline
Active Instances: 2
YOLO Tracking: 172ms
2026-06-25 13:03:26
ID:12 man 53.6% (224ms)
ID:4 man 57.8
VHR
/home/user/Downlo...
(no subject) - vempa...
user@user: ~/Downl...
Production Body-Sur...
13:03

---

6. Conclusion & Future Scope

The project successfully demonstrates a highly optimized, edge-ready surveillance pipeline. By structurally separating the detection logic from the classification logic using Python's threading and queue modules, the system guarantees a non-blocking real-time viewing experience. The implementation of a dynamic fallback mechanism (using the upper 40% of a crop when facial extraction fails) further proves the system's robustness in imperfect, real-world camera angles.

Future Scope

1. **Batch Processing:** Upgrading the consumer thread to process tensor batches rather than individual crops to maximize GPU throughput.
2. **Cloud Telemetry:** Integrating an API to push the collected demographic data (time, ID, predicted gender) to a cloud database for long-term behavioral analytics.
3. **Expanded Demographics:** Retraining the efficient_gender_model to include multi-class demographic estimations, such as age ranges or accessory detection (e.g., wearing a mask/glasses).

Distance (Approx)	Gender	Accuracy	YOLO inference	Classification Inference
2FT	Male	98.0%	76ms	409ms
5FT,9FT(towards right)	Male	53.6%,57.8%	163ms	224ms
15FT(straight)	Male	90.0%	48ms	395ms
15FT(towards left)	Male,Female	98.0%,89.7%	59ms	409ms,410ms
15FT(straight)	Female	90.0%,81.0%	82ms	372ms,231ms
9FT,14FT(slightly right)	Male,Female	100.0%,89.7%	169ms	410ms
9FT,13FT,17FT	Male,Female	90.1%,90.0%,80.0%	48ms	192ms,232ms,395ms

Project Code Link: [coruspus-999/Opencv-UOH]

Observation:

During live testing, the MEC-optimized pipeline successfully maintained a fluid video feed without frame drops by decoupling YOLO spatial tracking (76–163ms) from heavy demographic classifications (224–409ms). While the system achieved high predictive accuracy (90–100%) under optimal conditions, occasional latency spikes and confidence drops (53–58%) occurred due to real-world environmental constraints. Specifically, poor overhead lighting, sub-optimal camera angles, and partial subject coverage forced the AI to extrapolate incomplete geometric features, which temporarily degraded accuracy and increased CPU load. Despite these challenges, the system's dynamic fallback mechanics effectively stabilized the predictions, preventing UI flickering and resulting in a robust real-time monitoring

experience